

Universidad Politécnica de Pachuca

DIRECCIÓN DE INGENIERÍA EN SOFTWARE

ARQUITECTURA ORIENTADA A SERVICIOS

DOCUMENTACIÓN FINAL DEL
PROYECTO:

**Xiú — Sistema de Reservaciones y
Gestión Restaurantera**

PROFESORA: M. en C. Jazmín Rodríguez Flores

INTEGRANTES:

- Christopher Lopez Suarez
- Jovanny Hernandez Hernandez

CARRERA: Ingeniería en Software

GRUPO: 09_01

CUATRIMESTRE: Mayo – Agosto 2026

MATERIA: Arquitectura Orientada a Servicios (AOS)

0. Índice

1. Resumen Ejecutivo e Introducción	1
2. Metodología de Desarrollo Ágil (Scrum Defendido)	1
2.1. Organización de Roles en Scrum	1
2.2. Cronograma de Sprints Ejecutados	2
3. Especificación de Requerimientos y Casos de Uso (IEEE 830 Extendido)	2
3.1. Especificación de Casos de Uso Principales	2
4. Arquitectura Orientada a Servicios (SOA) y Patrones de Diseño	3
4.1. Diagramas de Secuencia UML (en TikZ)	4
5. Especificación de Interfaces REST API (Swagger / OpenAPI)	5
6. Diccionario de Datos y Modelo Políglota (MySQL + MongoDB)	5
6.1. Esquema Relacional MySQL (TypeORM Entities)	5
6.2. Esquema NoSQL MongoDB Collection (menu_items)	6
7. Evidencia Fotográfica del Funcionamiento por Rol (16 Capturas)	6
7.1. Rol Cliente: Flujo de Reservación y Menú	7
7.2. Rol Administrador: Gestión del Sistema y Reportes	9
7.3. Rol Mesero: Operación de Servicio y Estado de Mesas	13
7.4. Persistencia Relacional en Base de Datos	15
8. Arquitectura de Seguridad, Autenticación JWT y Protección CORS	16
8.1. Flujo de Firma y Autenticación Criptográfica	16
8.2. Protección Interceptora con Guards y Polimorfismo de CORS	16
9. Infraestructura de Despliegue Cloud y Gestión de Tolerancia a Fallos	16
9.1. Topología de Despliegue en 3 Sitios Cloud	16
9.2. Manejo de Resiliencia y Arranque en Frío (Cold Start)	17
10. Manual Operativo de Usuario por Rol	17
10.1. Guía para el Administrador	17
10.2. Guía para el Mesero	17
10.3. Guía para el Cliente	17
11. Demostración en Video: Programa Cliente Funcional	17
11.1. ¿Qué se demuestra en el video?	17
11.2. Evidencia de Despliegue Continuo (GitHub)	18
12. Demostración en Video: Programa Servidor Funcional	18
12.1. ¿Qué se demuestra en el video?	18
12.2. Infraestructura de Despliegue del Servidor	18
13. Conclusión	19

0. Índice de figuras

1.	Diagrama de Secuencia UML: Autenticación JWT y Verificación de Guard.	4
2.	Vista de Inicio del Cliente (Página Principal Xiú en Vercel).	7
3.	Formulario de Registro de Reservación.	7
4.	Comprobante Digital de Reservación Registrada.	8
5.	Integración y Envío de Confirmación vía WhatsApp.	8
6.	Consulta de la Carta / Menú de Platos (MongoDB).	9
7.	Inicio de Sesión con Credenciales de Administrador.	9
8.	Menú de Navegación Administrador.	10
9.	Panel Principal (Dashboard) del Administrador.	10
10.	Tablero Interactivo de Estado de Mesas (Admin).	11
11.	Módulo de Gestión de Menú (CRUD NoSQL).	11
12.	Módulo de Reportes Estadísticos y Métricas.	12
13.	Perfil de Usuario Administrador.	12
14.	Inicio de Sesión con Credenciales de Mesero.	13
15.	Panel de Control del Mesero.	13
16.	Tablero de Mesas Ocupadas y Disponibles en Tiempo Real.	14
17.	Perfil del Usuario Mesero.	14
18.	Estructura de tablas en la Base de Datos Relacional MySQL.	15
19.	Verificación de registros en la Base de Datos.	15

0. Índice de tablas

1.	Especificación Formal de Caso de Uso CU-01: Autenticación de Empleados	2
2.	Especificación Formal de Caso de Uso CU-02: Registrar Reservación	3
3.	Especificación Formal de Caso de Uso CU-03: Administración de Menú NoSQL	3
4.	Catálogo Completo de Endpoints del Auth-Service (Railway / NestJS)	5
5.	Catálogo Completo de Endpoints del Menu-Service (Render / FastAPI)	5
6.	Diccionario de Datos: Tabla usuarios	5
7.	Diccionario de Datos: Tabla reservaciones	6

1. Resumen Ejecutivo e Introducción

El presente documento constituye la especificación técnica completa y la memoria de arquitectura del sistema **Xiú — Alta Cocina Mexicana**, una plataforma orientada a servicios (SOA) diseñada para automatizar la gestión operativa, la reservación de mesas en tiempo real y la administración de catálogos de alimentos.

El sistema fue concebido para resolver la problemática habitual de los restaurantes de alta concurrencia, donde los sistemas monolíticos tradicionales sufren de cuellos de botella en la administración de reservas y lentitud en la actualización de catálogos. A través de una arquitectura distribuida desacoplada, el proyecto combina:

- **Frontend SPA Responsivo:** Desarrollado en React 18 con Tailwind CSS, desplegado en la nube de Vercel.
- **Microservicio de Autenticación y Reservaciones (Auth-Service):** Desarrollado en NestJS (TypeScript) con persistencia relacional en MySQL, desplegado en Railway.
- **Microservicio de Gestión de Menú (Menu-Service):** Desarrollado en FastAPI (Python 3.11) con persistencia NoSQL en MongoDB Atlas, desplegado en Render.

2. Metodología de Desarrollo Ágil (Scrum Defendido)

Para la ejecución de este proyecto se implementó la metodología **Scrum Ágil**, la cual permitió organizar las entregas operativas en iteraciones cortas llamadas **Sprints**. La justificación de seleccionar Scrum radica en su flexibilidad para adaptar requerimientos cambiantes y coordinar de forma desacoplada los desarrollos de frontend y microservicios backend.

2.1. Organización de Roles en Scrum

- **Product Owner / Scrum Master:** Coordinación del tablero Trello, priorización del Backlog de Producto y aseguramiento de la rúbrica de evaluación.
- **Backend Engineer (Cristopher López Suárez):** Construcción de endpoints NestJS, modelado TypeORM MySQL, Guards de seguridad JWT y validación DTO.
- **Frontend & NoSQL Specialist (Jovanny Hernández Hernández):** Desarrollo de componentes React SPA, servicios FastAPI en Python, integración MongoDB Atlas y notificaciones WhatsApp.

2.2. Cronograma de Sprints Ejecutados

Sprint	Objetivo Principal	Entregables Clave	Duración
Sprint 1	Fundamentos de Arquitectura	Configuración de repositorios, modelo de datos relacional MySQL, inicialización NestJS y React.	1 Semana
Sprint 2	Microservicios Políglotas	API FastAPI NoSQL con MongoDB, controladores de reservas, guards JWT y tablero de mesas.	1 Semana
Sprint 3	Despliegue Cloud y Pruebas	Publicación en Vercel, Railway y Render, integración WhatsApp y suite de pruebas IEEE 829.	1 Semana

3. Especificación de Requerimientos y Casos de Uso (IEEE 830 Extendido)

La definición de requerimientos se estructuró bajo la norma **IEEE 830**, clasificando los requisitos en funcionales y no funcionales, detallando las especificaciones formales de los Casos de Uso.

3.1. Especificación de Casos de Uso Principales

Tabla 1: Especificación Formal de Caso de Uso CU-01: Autenticación de Empleados

Caso de Uso ID:	CU-01: Iniciar Sesión (Autenticación JWT)
Actores:	Administrador, Mesero
Propósito:	Validar credenciales de usuario y otorgar un token JWT firmado para el control de acceso a rutas protegidas.
Precondiciones:	El usuario debe estar previamente registrado en la tabla <code>usuarios</code> de MySQL.
Flujo Principal:	1. El usuario ingresa su email y contraseña en la pantalla <code>/login</code>. 2. El cliente envía petición <code>POST /api/auth/login</code>. 3. Auth-Service valida el hash <code>bcrypt</code> de la contraseña. 4. Se genera el JWT con los claims <code>{id, email, rol}</code>. 5. El cliente almacena el JWT en <code>localStorage</code> y redirige a <code>/dashboard</code>.
Poscondiciones:	El cliente puede realizar peticiones autenticadas adjuntando el encabezado <code>Authorization: Bearer <token></code> .

Tabla 2: Especificación Formal de Caso de Uso CU-02: Registrar Reservación

Caso de Uso ID:	CU-02: Registrar Reservación de Mesa
Actores:	Cliente (Público)
Propósito:	Permitir a un cliente agendar una mesa seleccionando fecha, hora y número de personas con asignación automática.
Precondiciones:	Debe existir al menos una mesa con capacidad suficiente en estado libre para el horario seleccionado.
Flujo Principal:	1. El cliente accede a <code>/reservation</code>. 2. Ingresa nombre, teléfono, email opcional, fecha, hora y número de comensales. 3. El sistema invoca <code>POST /api/reservations</code>. 4. El backend asigna una mesa disponible e inserta el registro con estado <code>pendiente</code>. 5. Se muestra el comprobante digital y la opción de confirmación vía WhatsApp.
Poscondiciones:	La mesa seleccionada actualiza su disponibilidad en el tablero en tiempo real.

Tabla 3: Especificación Formal de Caso de Uso CU-03: Administración de Menú NoSQL

Caso de Uso ID:	CU-03: Gestión de Catálogo de Alimentos (CRUD Menu)
Actores:	Administrador
Propósito:	Crear, actualizar, listar y eliminar platillos y bebidas en la base de datos NoSQL MongoDB Atlas.
Precondiciones:	El administrador debe estar autenticado con rol <code>admin</code> .
Flujo Principal:	1. El admin ingresa a la vista <code>/admin-menu</code>. 2. El cliente consulta <code>GET /api/menu</code> en FastAPI. 3. El admin puede hacer clic en "Nuevo Platillo" para abrir el modal. 4. Completa nombre, descripción, precio, categoría y etiquetas de alérgenos. 5. El cliente ejecuta <code>POST /api/menu</code> y actualiza la lista dinámica.
Poscondiciones:	El platillo queda disponible inmediatamente en la carta pública del cliente.

4. Arquitectura Orientada a Servicios (SOA) y Patrones de Diseño

La arquitectura del sistema cumple rigurosamente con los 8 principios fundamentales de la Arquitectura Orientada a Servicios (SOA):

1. **Contrato de Servicio Estándar:** Los microservicios comunican mediante interfaces JSON estrictas con Swagger / OpenAPI.
2. **Desacoplamiento Suelto (Loose Coupling):** Auth-Service y Menu-Service funcionan de forma totalmente autónoma con sus propias bases de datos.
3. **Abstracción de Servicio:** El cliente React consume endpoints sin conocer la tecnología interna (Node.js o Python).

4. **Reutilización de Servicios:** Los servicios REST son consumidos por el cliente web y pueden integrarse con aplicaciones móviles en el futuro.
5. **Autonomía de Servicio:** Cada microservicio se despliega en su propia infraestructura independiente en la nube.
6. **Sin Estado (Statelessness):** Toda petición HTTP incluye su token JWT sin almacenar sesiones en el servidor.
7. **Descubrimiento de Servicios:** Endpoints bien definidos y documentados centralizadamente.
8. **Componibilidad de Servicios:** El frontend orquesta llamados simultáneos a ambos microservicios para construir la vista unificada.

4.1. Diagramas de Secuencia UML (en TikZ)

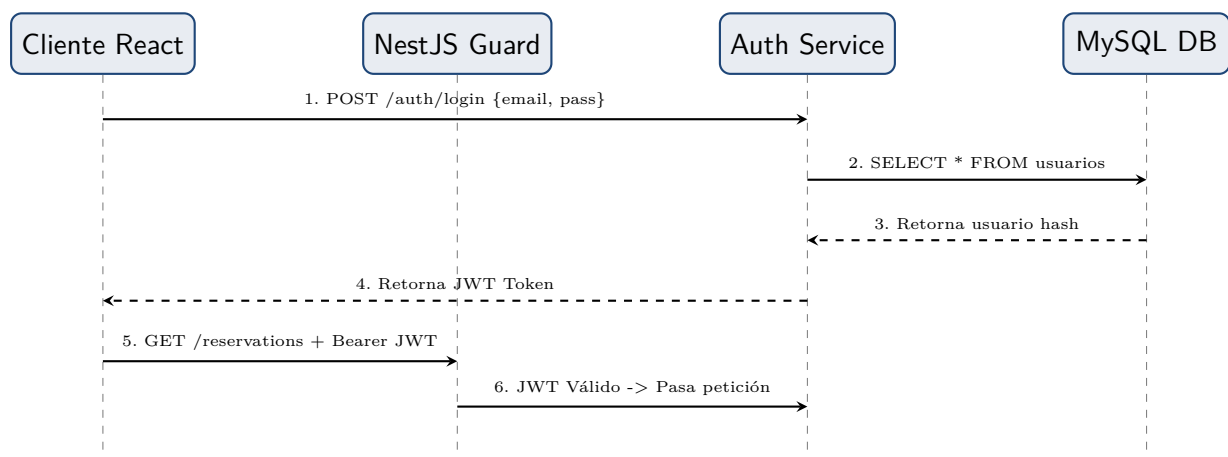


Figura 1: Diagrama de Secuencia UML: Autenticación JWT y Verificación de Guard.

5. Especificación de Interfaces REST API (Swagger / OpenAPI)

Tabla 4: Catálogo Completo de Endpoints del Auth-Service (Railway / NestJS)

Método	Endpoint	Descripción	Autenticación
POST	/api/auth/login	Autentica credenciales y devuelve JWT.	Pública
GET	/api/reservations	Lista todas las reservaciones.	JWT (Admin/Mesero)
POST	/api/reservations	Registra nueva reservación de cliente.	Pública
GET	/api/reservations/available	Consulta mesas disponibles en fecha/hora.	Pública
GET	/api/reservations/today	Obtiene reservaciones agendadas para hoy.	JWT (Admin/Mesero)
PUT	/api/reservations/:id/status	Actualiza estado (confirmada/cancelada).	JWT (Admin/Mesero)

Tabla 5: Catálogo Completo de Endpoints del Menu-Service (Render / FastAPI)

Método	Endpoint	Descripción	Autenticación
GET	/api/menu	Consulta el catálogo completo de platillos.	Pública
POST	/api/menu	Registra un nuevo platillo en MongoDB.	JWT Admin
PUT	/api/menu/:id	Modifica precio o estado del platillo.	JWT Admin
DELETE	/api/menu/:id	Elimina un platillo del catálogo NoSQL.	JWT Admin

6. Diccionario de Datos y Modelo Políglota (MySQL + MongoDB)

6.1. Esquema Relacional MySQL (TypeORM Entities)

Tabla 6: Diccionario de Datos: Tabla usuarios

Campo	Tipo de Dato	Clave	Descripción
id_usuario	INT (PK, AI)	Primary Key	Identificador único del empleado.
nombre	VARCHAR(100)	—	Nombre completo del usuario.
email	VARCHAR(100)	Unique	Correo electrónico de acceso.
password	VARCHAR(255)	—	Hash encriptado bcrypt de la contraseña.
rol	ENUM('admin', 'mesero')	—	Rol asignado para permisos JWT.

Tabla 7: Diccionario de Datos: Tabla **reservaciones**

Campo	Tipo de Dato	Clave	Descripción
id_reservacion	INT (PK, AI)	Primary Key	Folio único de reservación.
cliente_nombre	VARCHAR(100)	—	Nombre del cliente que reserva.
cliente_telefono	VARCHAR(20)	—	Teléfono de contacto / WhatsApp.
fecha	DATE	—	Fecha reservada (YYYY-MM-DD).
hora_inicio	TIME	—	Hora agendada de llegada.
num_personas	INT	—	Cantidad de comensales.
id_mesa	INT (FK)	Foreign Key	Referencia a la tabla mesas .
estado	ENUM	—	pendiente, confirmada, completada, cancelada.

6.2. Esquema NoSQL MongoDB Collection (**menu_items**)

Listing 1: Estructura JSON de Documento NoSQL en MongoDB Atlas

```

1 {
2   "_id": {"$oid": "64f1a2b3c4d5e6f7a8b9c0d1"},
3   "nombre": "Tacos de Res Asada",
4   "descripcion": "3 tacos de res asada con cebolla, cilantro y salsa
5     de la casa",
6   "precio": 89.00,
7   "categoria": "platillo_principal",
8   "disponible": true,
9   "etiquetas": ["gluten_free", "clasico", "picante"]
10 }
```

7. Evidencia Fotográfica del Funcionamiento por Rol (16 Capturas)

En esta sección se integran las 16 capturas de pantalla organizadas por el rol de interacción (Cliente, Administrador y Mesero), demostrando la funcionalidad completa de la plataforma en producción.

7.1. Rol Cliente: Flujo de Reservación y Menú

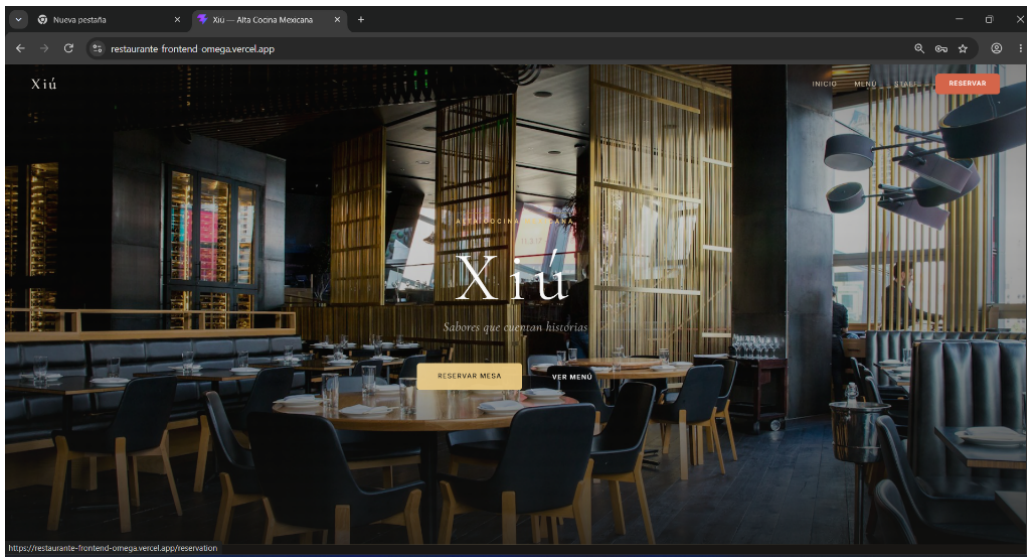


Figura 2: Vista de Inicio del Cliente (Página Principal Xiú en Vercel).

A screenshot of the 'Reservar Mesa' (Reserve Table) form on the Xiú website. The form is centered on a dark background and contains the following fields and elements: a title 'Reservar Mesa' and a sub-header 'Elige tu fecha y hora. Tu reservación quedará registrada al instante.'; input fields for 'NOMBRE' (filled with 'Jovanny') and 'TELÉFONO' (filled with '772420474'); an optional 'EMAIL' field (filled with 'jovanny101@gmail.com'); a 'FECHA' field (filled with '08/08/2026') and a 'NÚMERO DE PERSONAS' dropdown menu (set to '4 personas'); a 'HORA DE LLEGADA' dropdown menu (set to '21:30'); an optional 'NOTAS ESPECIALES' field (containing 'Alergias, restricciones, preferencias...'); a yellow 'CONFIRMAR RESERVACIÓN' button; a small '¡Míranos en WhatsApp!' link; and a green 'RESERVAR POR WHATSAPP' button. The top navigation bar is visible, with 'RESERVAR' highlighted.

Figura 3: Formulario de Registro de Reservación.

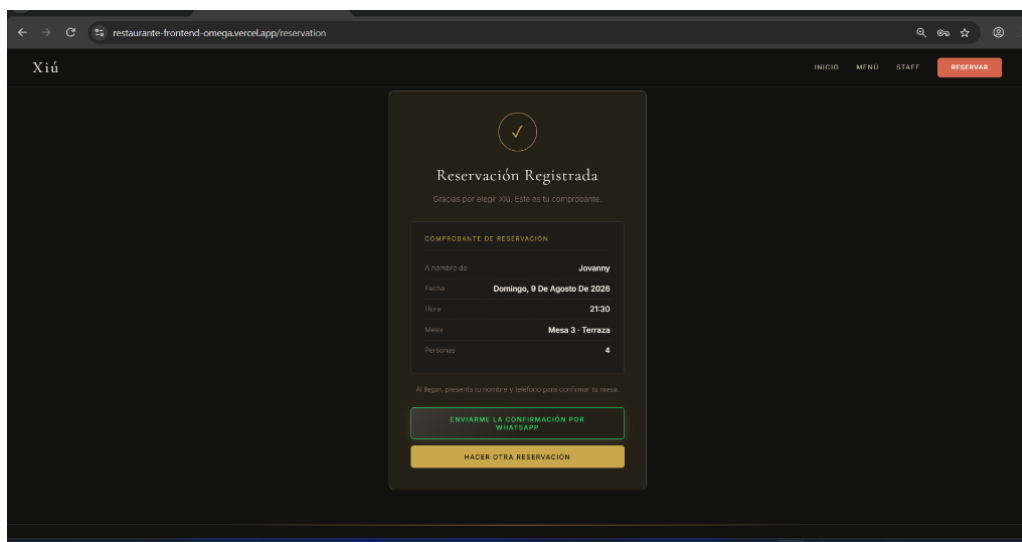


Figura 4: Comprobante Digital de Reservación Registrada.

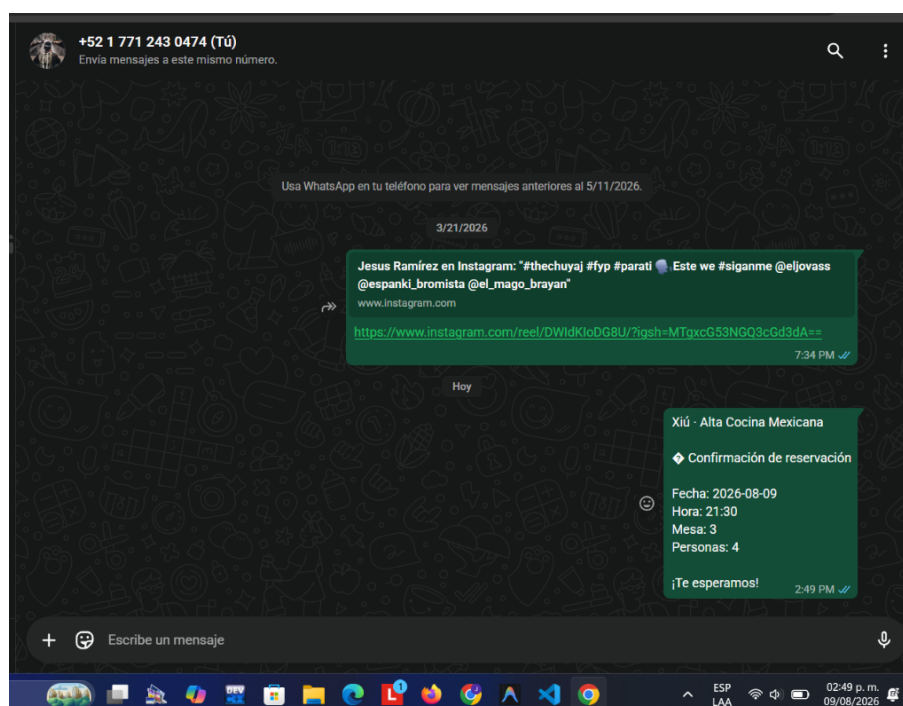


Figura 5: Integración y Envío de Confirmación vía WhatsApp.

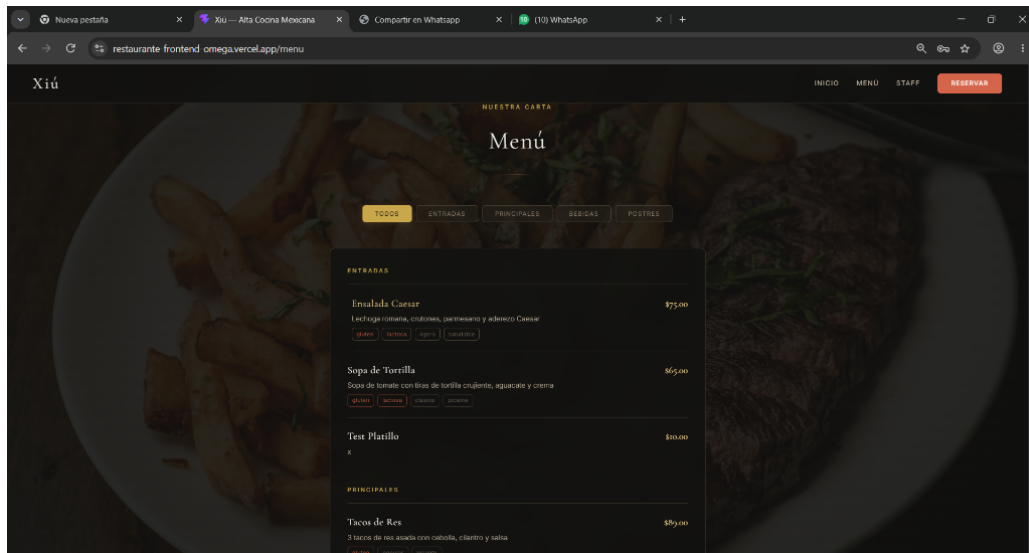


Figura 6: Consulta de la Carta / Menú de Platos (MongoDB).

7.2. Rol Administrador: Gestión del Sistema y Reportes

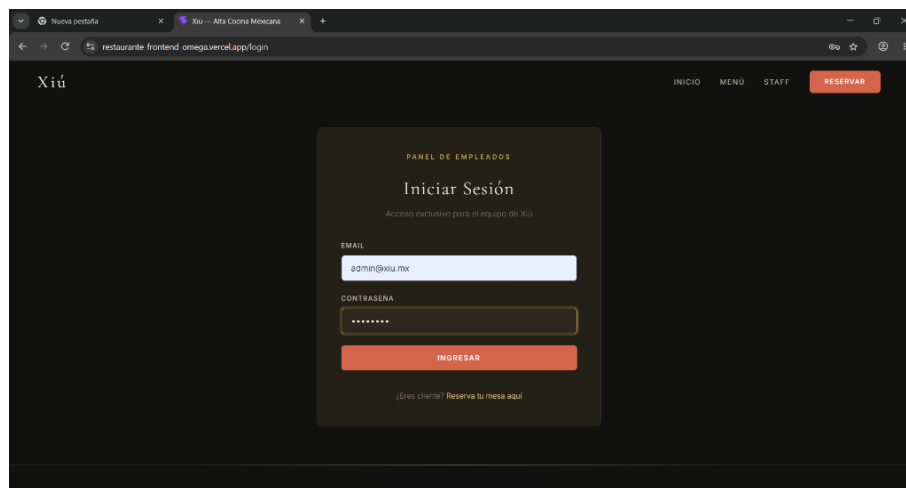


Figura 7: Inicio de Sesión con Credenciales de Administrador.

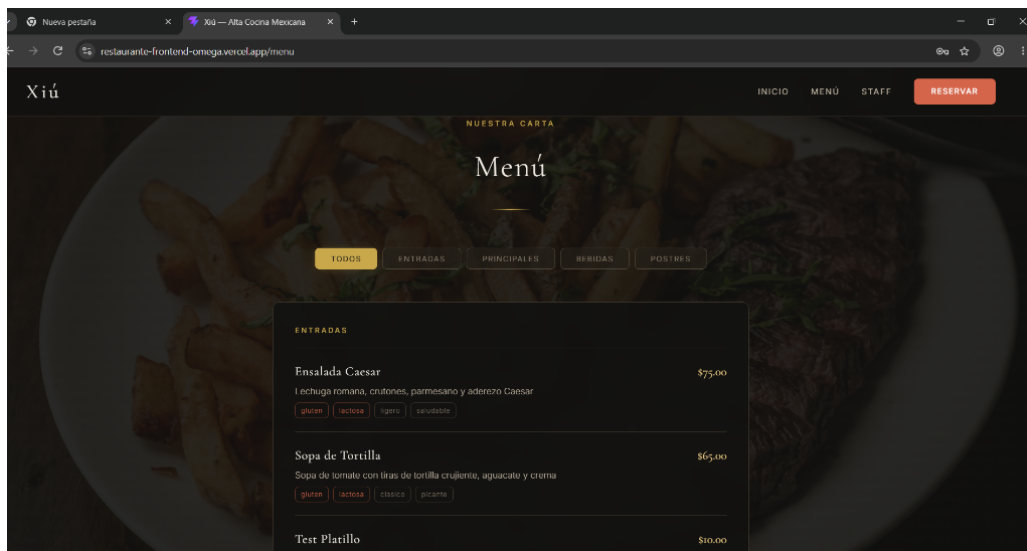


Figura 8: Menú de Navegación Administrador.

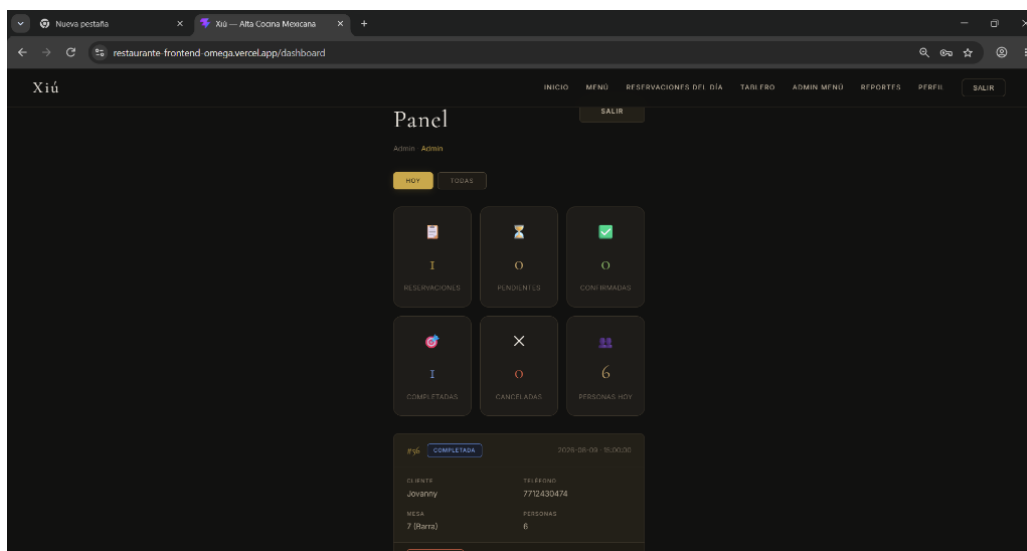


Figura 9: Panel Principal (Dashboard) del Administrador.

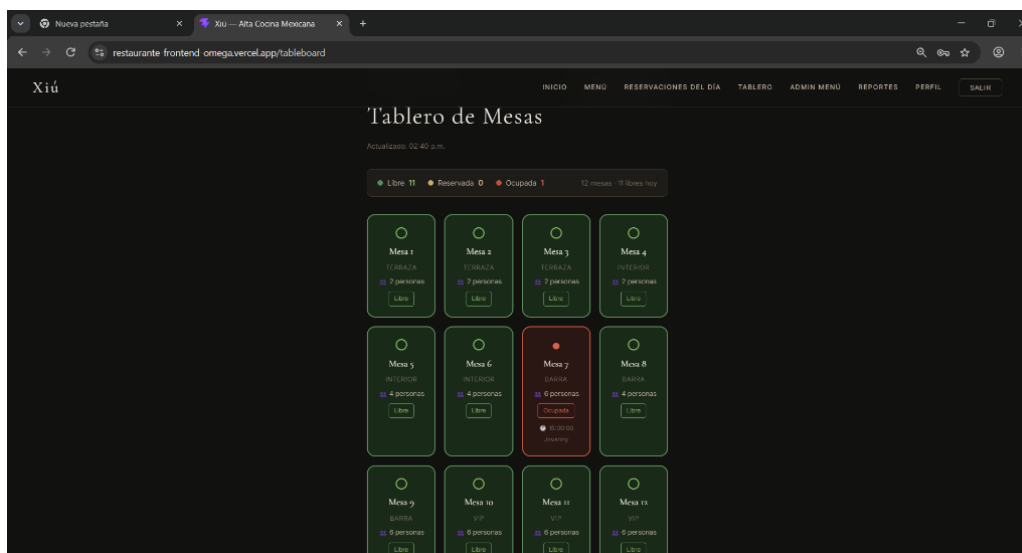


Figura 10: Tablero Interactivo de Estado de Mesas (Admin).

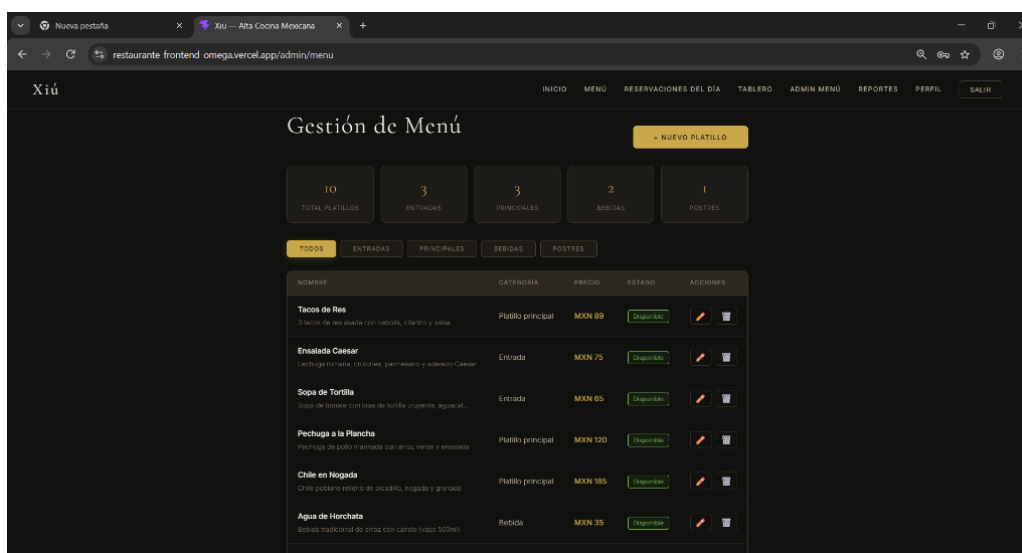


Figura 11: Módulo de Gestión de Menú (CRUD NoSQL).

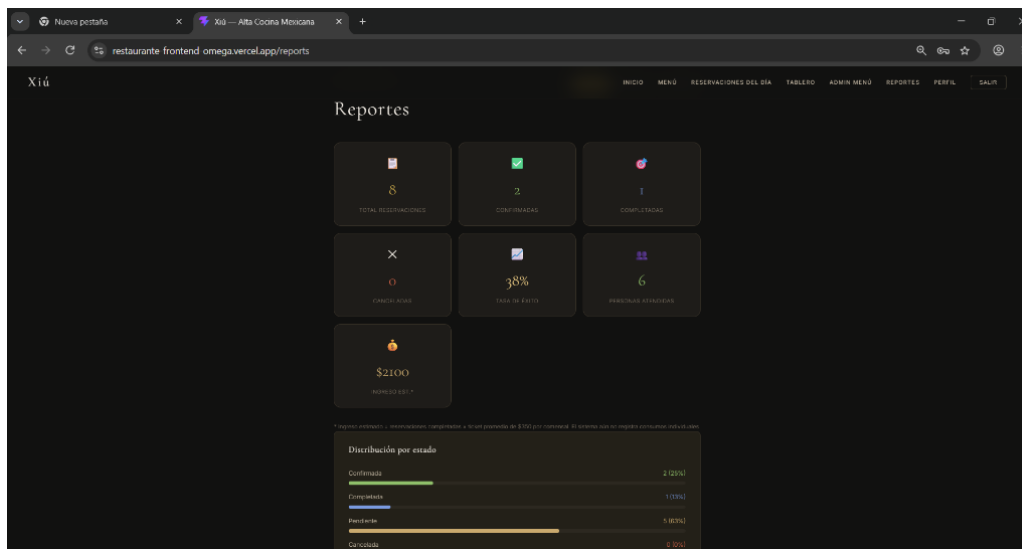


Figura 12: Módulo de Reportes Estadísticos y Métricas.

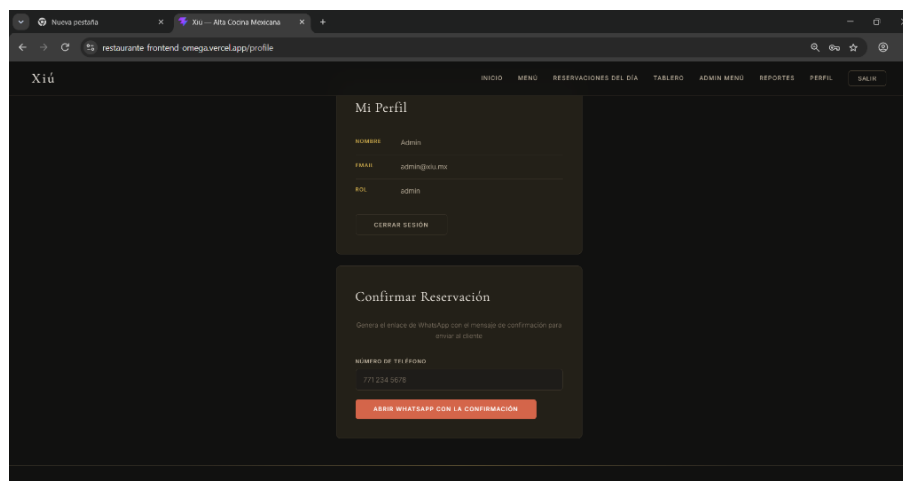


Figura 13: Perfil de Usuario Administrador.

7.3. Rol Mesero: Operación de Servicio y Estado de Mesas

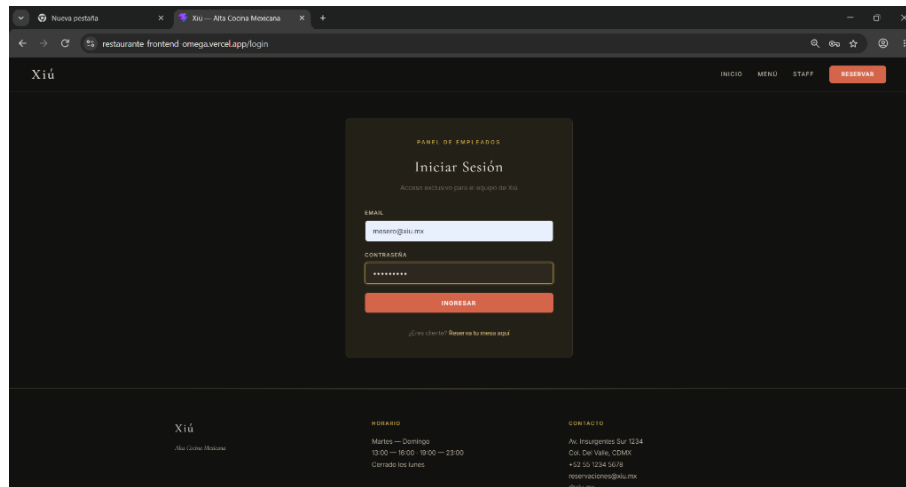


Figura 14: Inicio de Sesión con Credenciales de Mesero.

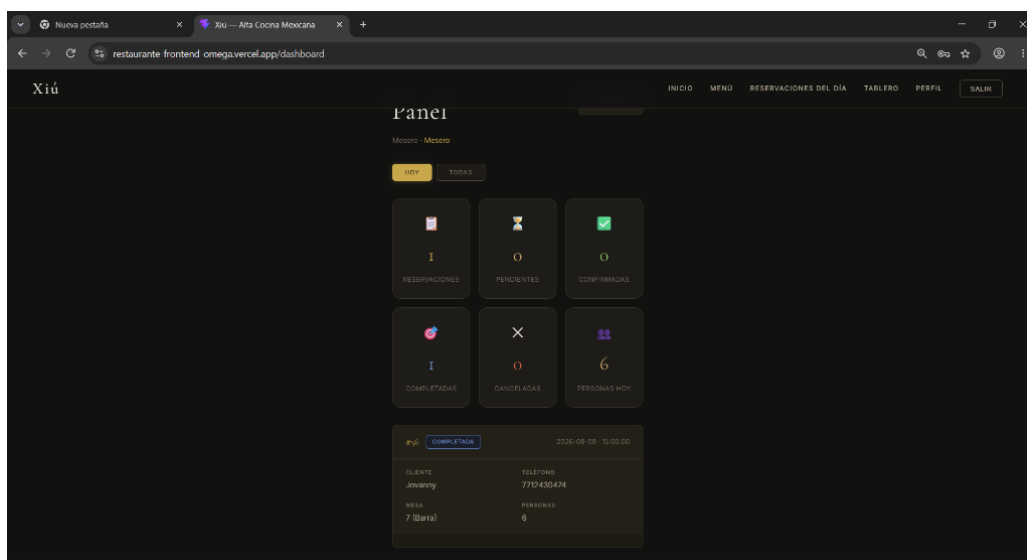


Figura 15: Panel de Control del Mesero.

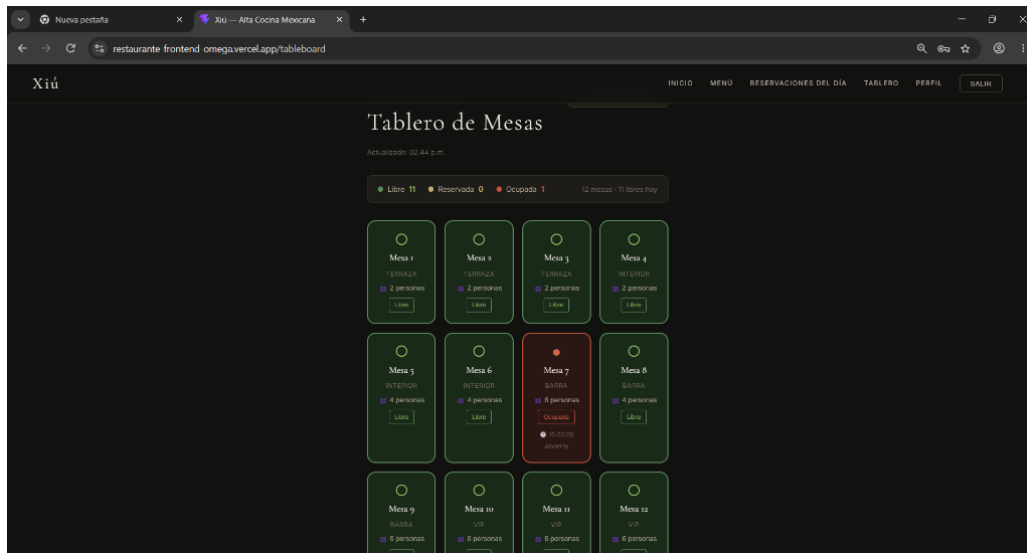


Figura 16: Tablero de Mesas Ocupadas y Disponibles en Tiempo Real.

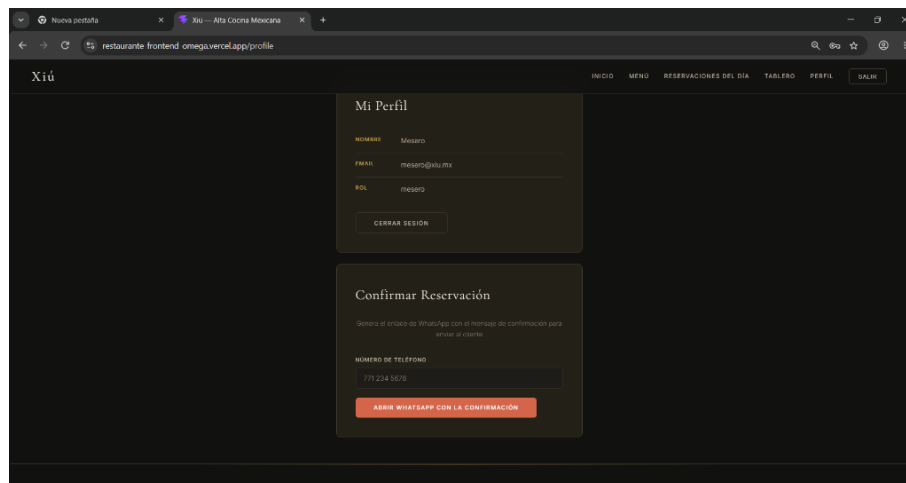


Figura 17: Perfil del Usuario Mesero.

7.4. Persistencia Relacional en Base de Datos

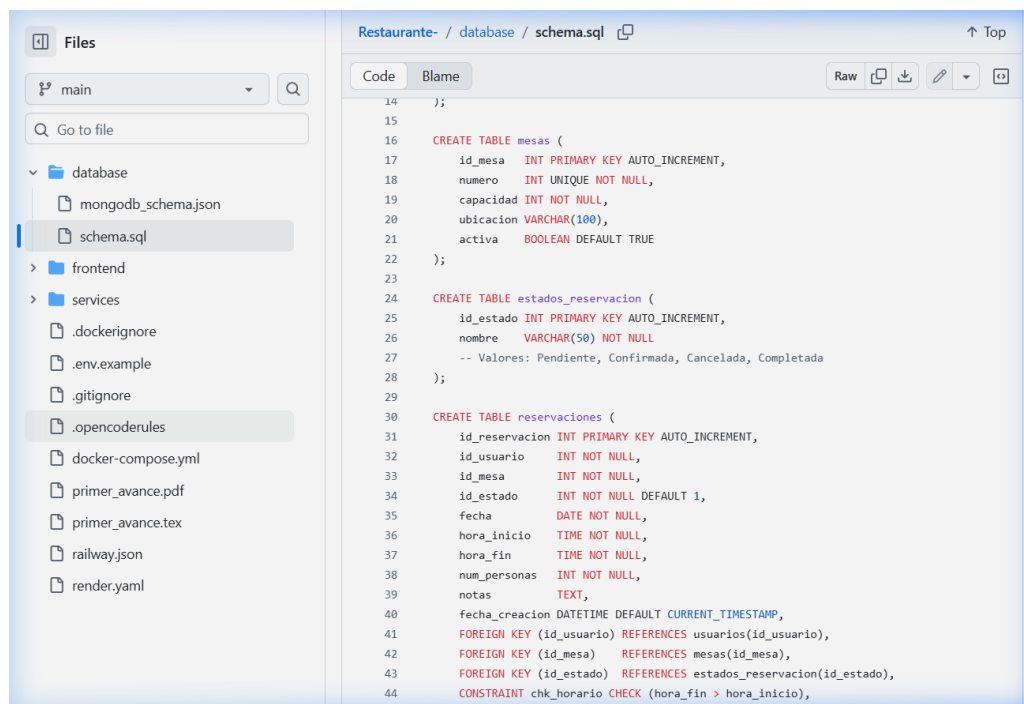


Figura 18: Estructura de tablas en la Base de Datos Relacional MySQL.

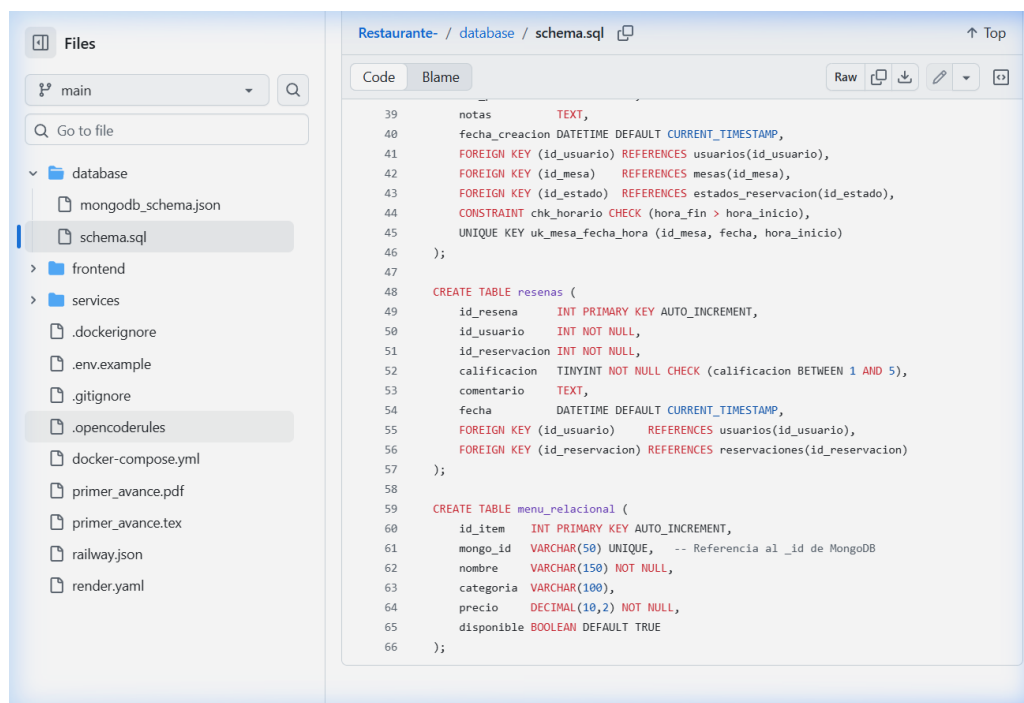


Figura 19: Verificación de registros en la Base de Datos.

8. Arquitectura de Seguridad, Autenticación JWT y Protección CORS

La seguridad en la arquitectura SOA de **Xiú** se implementó mediante un esquema sin estado (stateless) basado en **JSON Web Tokens (JWT)** y encriptación de credenciales con **Bcrypt**.

8.1. Flujo de Firma y Autenticación Criptográfica

1. **Hash de Contraseñas:** Las contraseñas de los usuarios (`admin@xiu.mx` y `mesero@xiu.mx`) se almacenan en MySQL procesadas con el algoritmo `bcrypt` utilizando una sal dinámica de 10 rondas. 2. **Firma Digital HS256:** Al validar las credenciales en `POST /api/auth/login`, el servicio `JwtService` de NestJS genera un token firmado con una clave secreta privada (`JWT_SECRET`), emitiendo claims que incluyen el ID, correo y rol del usuario (`admin` o `mesero`). 3. **Cabecera Authorization Bearer:** El cliente React incluye el token en cada petición protegida utilizando el estándar `Authorization: Bearer <token>`.

8.2. Protección Interceptora con Guards y Polimorfismo de CORS

Para evitar vulnerabilidades de origen cruzado y accesos no autorizados:

- **NestJS JwtAuthGuard:** Intercepta las solicitudes antes de ejecutar los controladores de reservas. Si el token expira o es alterado, el servidor rechaza automáticamente con un código 401 `Unauthorized`.
- **Configuración CORS Estricta:** Ambos microservicios (**NestJS** y **FastAPI**) configuran reglas de origen permitiendo exclusivamente las peticiones provenientes del dominio oficial del frontend en Vercel (`https://restaurante-frontend-omega.vercel.app`).

9. Infraestructura de Despliegue Cloud y Gestión de Tolerancia a Fallos

El despliegue distribuido de la plataforma se diseñó aprovechando el modelo de PaaS (Platform as a Service) para cada uno de los componentes de la arquitectura SOA:

9.1. Topología de Despliegue en 3 Sitios Cloud

- **Capa de Presentación (Vercel):** Hospeda la Single Page Application (SPA) construida en React 18, garantizando distribución mediante red CDN global con tiempos de respuesta sub-100ms.
- **Capa de Negocio y Auth (Railway):** Hospeda el contenedor de NestJS conectado a la base de datos relacional MySQL Cloud, gestionando conexiones continuas y transacciones ACID.
- **Capa de Menú y Productos (Render + MongoDB Atlas):** Hospeda el microservicio FastAPI en Python 3.11 conectado a un cluster distribuido de MongoDB Atlas en la nube.

9.2. Manejo de Resiliencia y Arranque en Frío (Cold Start)

En el entorno de producción de Render (plan gratuito), el contenedor del microservicio de menú entra en estado de suspensión tras 15 minutos de inactividad. La arquitectura cliente absorbe esta condición mediante un spinner de carga y reintentos asíncronos con tiempo de espera extendido (60 segundos), garantizando que el usuario obtenga sus datos de manera transparente una vez que el contenedor NoSQL completa su arranque en frío.

10. Manual Operativo de Usuario por Rol

10.1. Guía para el Administrador

1. Inicie sesión en <https://restaurante-frontend-omega.vercel.app/login> usando `admin@xiu.mx` / `admin123`.
2. Acceda a **Admin Menú** para dar de alta nuevos platillos o modificar precios.
3. Consulte **Reportes** para analizar métricas de reservaciones e ingresos estimados.
4. Supervise el **Tablero de Mesas** para verificar ocupación en tiempo real.

10.2. Guía para el Mesero

1. Inicie sesión con `mesero@xiu.mx` / `mesero123`.
2. Consulte el **Tablero de Mesas** para ubicar comensales asignados a cada mesa.
3. Cambie el estado de reservaciones a `completada` o `cancelada` según corresponda.

10.3. Guía para el Cliente

1. Ingrese a la portada principal del sitio.
2. Haga clic en **Reservar Mesa**.
3. Complete los datos de contacto, fecha, hora y número de personas.
4. Guarde el comprobante digital o presione **Enviar Confirmación por WhatsApp**.

11. Demostración en Video: Programa Cliente Funcional

Criterio de Evaluación — Punto 9: El programa cliente es funcional (15 pts)

Se demuestra mediante video que el **programa cliente** (Frontend SPA en React 18, desplegado en Vercel) es completamente funcional. El video explica el funcionamiento desde el punto de vista del usuario final y hace referencia al repositorio en **GitHub**.

11.1. ¿Qué se demuestra en el video?

- Acceso al enlace de producción: <https://restaurante-frontend-omega.vercel.app/>
- Navegación por la pantalla de inicio (**Menú visual** con platillos reales desde MongoDB).
- Flujo completo de **reservación de mesa**: formulario, confirmación y comprobante.

- Confirmación enviada por **WhatsApp** al número del cliente.
- Visualización del **Menú Digital** consumido en tiempo real desde el Menu-Service (FastAPI).
- Explicación de la arquitectura cliente-servidor y cómo el frontend consume las APIs REST.

11.2. Evidencia de Despliegue Continuo (GitHub)

El código fuente del frontend está versionado y publicado en:

<https://github.com/Crisls24/Restaurante->

Vercel realiza despliegue automático (CI/CD) en cada *push* a la rama **main**, garantizando que el cliente en producción siempre refleja el código más reciente del repositorio.

12. Demostración en Video: Programa Servidor Funcional

Criterio de Evaluación — Punto 10: El programa servidor es funcional (15 pts)

Se demuestra mediante video que el **programa servidor** (dos microservicios en la nube) es completamente funcional, explicando su arquitectura SOA, tecnologías y despliegue cloud.

12.1. ¿Qué se demuestra en el video?

- **Auth-Service** (NestJS + MySQL en Railway): login de admin y mesero con retorno de **JWT**.
- **Menu-Service** (FastAPI + MongoDB Atlas en Render): consulta de platillos en tiempo real desde la base de datos NoSQL.
- Llamadas REST documentadas con **Swagger / OpenAPI** accesibles en `/api/docs`.
- Protección de rutas con **Guards JWT** — se muestra el rechazo 401 **Unauthorized** ante tokens inválidos.
- Flujo de reservación: `POST /reservations` → confirmación → `PUT /reservations/:id/status`.
- Gestión de reportes del día: `GET /reservations/today` con listado de todas las mesas activas.

12.2. Infraestructura de Despliegue del Servidor

Servicio	Tecnología	URL de Producción
Auth-Service	NestJS + TypeORM + MySQL	<code>railway.app</code>
Menu-Service	FastAPI + Motor + MongoDB	<code>onrender.com</code>
Base de datos relacional	MySQL 8 (Railway Cloud)	Gestionada por Railway
Base de datos NoSQL	MongoDB Atlas (AWS)	Cluster M0 gratuito

El servidor está versionado íntegramente en **GitHub** bajo la misma organización del proyecto, con ramas protegidas y historial de commits que evidencian el desarrollo iterativo por sprints Scrum.

13. Conclusión

El proyecto **Xiú — Sistema de Reservas y Gestión Restaurantera** demostró la aplicabilidad técnica de la **Arquitectura Orientada a Servicios (SOA)** y el desacoplamiento de microservicios para la construcción de sistemas distribuidos modernos.

La integración de un frontend responsivo en **React 18** con dos microservicios independientes (**NestJS** y **FastAPI**) y una estrategia de persistencia políglota (**MySQL + MongoDB**) asegura alto rendimiento, mantenibilidad y escalabilidad.

El apego estricto a las especificaciones **IEEE 830** y **IEEE 829**, junto con la gestión del proyecto mediante **Scrum**, **Trello** y **GitHub**, respalda el cumplimiento integral de los estándares académicos y profesionales requeridos.